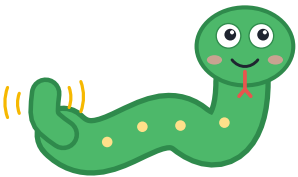
Lecture 3

Part 1: Intro and Python Basics

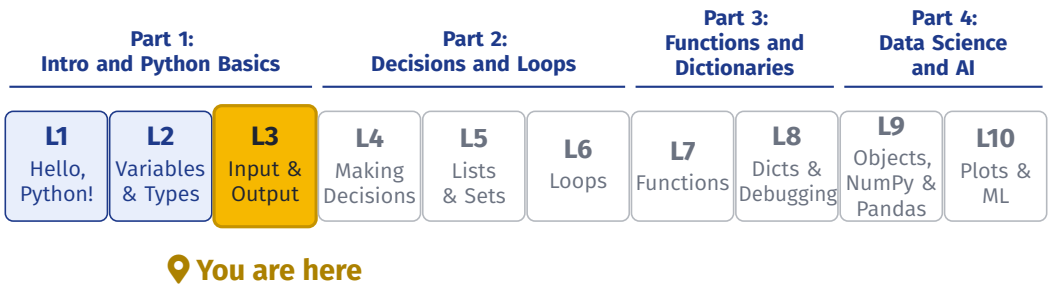
Input, Output & Conditions

Programs that ask, answer, and make their first decision



Dror Jacoby
Tel Aviv University

Where we are


Part 1:
Intro and Python Basics
Part 2:
Decisions and Loops
Part 3:
Functions and
Dictionaries
Part 4:
Data Science
and AI
L1
Hello,
Python!
L2
Variables
& Types
L3
Input &
Output
L4
Making
Decisions
L5
Lists
& Sets
L6
Loops
L7
Functions
L8
Dicts &
Debugging
L9
Objects,
NumPy &
Pandas
L10
Plots &
ML
You are here

Looking back: lectures 1 and 2

✓ Lecture 1: Hello, Python!

- A program is clear steps for a very literal intern
- Every computer: input, processing, output (and memory)
- LLMs predict the next token. Useful, but check them
- In Colab, `print(...)` shows text and results

✓ Lecture 2: Variables & Types

- A variable is a labeled box. `=` means “store”
- Types: `int`, `float`, `str`, `bool`. Ask `type()`
- `+ - * / // % **`. `/` always gives a float
- Text can do `+` (glue) and `*` (repeat)

Warm-up: lectures 1 and 2 in one program

```
price = 14
cups = 3
total = price * cups
print("Total:", total)
print(type(total), type(total / 2))
print(total // 4, total % 4)
print("ha" * 3)
```

Warm-up: lectures 1 and 2 in one program

```
price = 14
cups = 3
total = price * cups
print("Total:", total)
print(type(total), type(total / 2))
print(total // 4, total % 4)
print("ha" * 3)
```

OUTPUT

```
Total: 42
<class 'int'> <class
'float'>
10 2
hahaha
```

Quick check: predict the output

```
x = 7
x = x + 3
print(x / 2, x // 3, x % 3)
print("7" + "3")
```

Quick check: predict the output

```
x = 7
x = x + 3
print(x / 2, x // 3, x % 3)
print("7" + "3")
```

OUTPUT

```
5.0 3 1
73
```

Quick check: predict the output

```
x = 7
x = x + 3
print(x / 2, x // 3, x % 3)
print("7" + "3")
```

OUTPUT

```
5.0 3 1
73
```

- `x = x + 3`: the box now holds `10`
- `/` gives a float, `//` and `%` split whole numbers
- Quotes make it text: `+` glues, it does not add

Programs that talk back

- The ATM asks for your PIN
- The food app asks for your address

Programs that talk back

- The ATM asks for your PIN
- The food app asks for your address
- Our programs so far only **talk to themselves**
- Today: they **ask**, **listen**, and **answer**



Today's plan

◎ By the end of today you can...

- Print nicely and ask the user questions
- Turn text into numbers, and build sentences with f-strings
- Let your program choose with `if` / `else`



Today's outline

1. **Talking with the user**
   `print` with several values, asking with `input()`
2. **Converting types and f-strings**
   `int()`, `float()`, `str()`, f-strings and `:.2f`
3. **Intro to conditions**
   All six comparisons, `=` vs `==`, chaining, `if` / `else`, indentation



Every topic: examples, **Try it!** and **Predict the output**. At the end: **Bug Zoo**, **AI Corner**, **Recap**.

Why today matters

Why learn this?

- A program that can't talk to people is a program nobody uses
- Clear output = a receipt, a message, a report people understand
- The first `if`: your program starts making choices

Where you will see it

- Forms, sign-ups, chatbots
- Receipts, bank messages, delivery updates
- "Free shipping over 200 ILS"





Topic 1

Talking with the user

Lecture 3 • Input, Output & Conditions



TODAY

1. **Talking with the user**
2. Converting types and f-strings
3. Intro to conditions

`print` can show several things

```
print("Coffee costs", 14, "ILS")
print("Dana", "Yossi", "Noa")
```



`print` can show several things

```
print("Coffee costs", 14, "ILS")
print("Dana", "Yossi", "Noa")
```

OUTPUT

```
Coffee costs 14 ILS
Dana Yossi Noa
```



`print` can show several things

```
print("Coffee costs", 14, "ILS")
print("Dana", "Yossi", "Noa")
```

OUTPUT

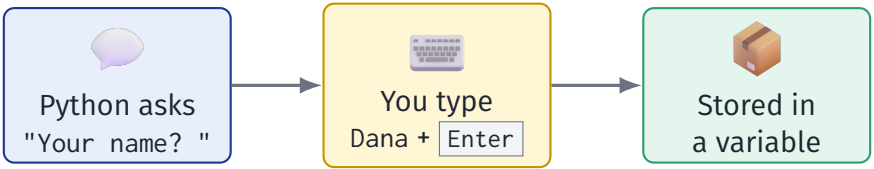
```
Coffee costs 14 ILS
Dana Yossi Noa
```

- Separate the values with commas
- Python puts a space between them
- Text and numbers mix fine here

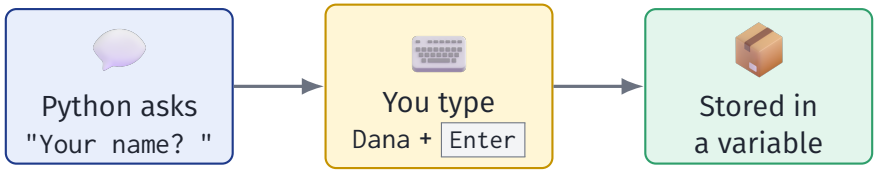


ILS is the shekel's official ISO 4217 code (like USD, EUR). In everyday English you'll also see NIS.

`input()`: the program asks and waits



`input()`: the program asks and waits



- The program **stops** until you press `Enter`
- Whatever you typed goes into a variable

Your first conversation

```
name = input("Your name? ")
print("Hi", name)
```

Your first conversation

```
name = input("Your name? ")
print("Hi", name)
```

OUTPUT

```
Your name? Dana
Hi Dana
```

(the user typed Dana)

Your first conversation

```
name = input("Your name? ")
print("Hi", name)
```

OUTPUT

```
Your name? Dana
Hi Dana
```

(the user typed Dana)

Tip

End the question with a space: `"Your name?␣"`. Otherwise the answer sticks to it.

Surprise: `input()` always gives text

```
num = input("Pick a number: ")
print(type(num))
print(num + num)
```

Surprise: `input()` always gives text

```
num = input("Pick a number: ")
print(type(num))
print(num + num)
```

OUTPUT

```
Pick a number: 5
<class 'str'>
55
```

Surprise: `input()` always gives text

```
num = input("Pick a number: ")
print(type(num))
print(num + num)
```

OUTPUT

```
Pick a number: 5
<class 'str'>
55
```



You typed `5`. Python sees the *text* `"5"`.

Try it!

Try it in Colab notebooks/L03/L03_01_print_and_input.ipynb

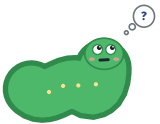
1. Print your name and your age in **one** `print`
2. Ask for the user's favorite food, then print it back
3. Ask for a number and print it twice. Why `55`?



Predict the output

Predict the output!

```
x = input("x? ")   # you type 4
print(x * 3)
print("x is", x)
```



Predict the output

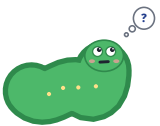
Predict the output!

```
x = input("x? ")   # you type 4
print(x * 3)
print("x is", x)
```

Answer

```
x? 4
444
x is 4
```

`x` is the text `"4"`, and text times 3 repeats it.



Topic 2

Converting types and f-strings

Lecture 3 • Input, Output & Conditions



TODAY

1 Talking with the user

2 Converting types and f-strings

3 Intro to conditions

Converting between types

Function	Turns a value into…	Example
`int()`	a whole number	`int("20")` → `20`
`float()`	a decimal number	`float("3.5")` → `3.5`
`str()`	text	`str(20)` → `"20"`



Converting between types

Function	Turns a value into…	Example
`int()`	a whole number	`int("20")` → `20`
`float()`	a decimal number	`float("3.5")` → `3.5`
`str()`	text	`str(20)` → `"20"`

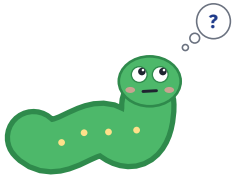


In real life

Like changing shekels to dollars: same value, a new form.

`int()` chops, it does not round

```
print(int(3.9))
print(round(3.9))
print(float(7))
```



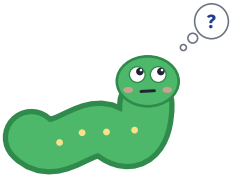
3.9 hours of sleep is still not 4 hours.

`int()` chops, it does not round

```
print(int(3.9))
print(round(3.9))
print(float(7))
```

OUTPUT

```
3
4
7.0
```



3.9 hours of sleep is still not 4 hours.

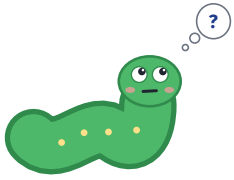
`int()` chops, it does not round

```
print(int(3.9))
print(round(3.9))
print(float(7))
```

OUTPUT

```
3
4
7.0
```

`int(3.9)` throws away everything after the dot. Want to round? Use `round()`.



3.9 hours of sleep is still not 4 hours.

Fix: convert the answer right away

```
age = int(input("Your age? "))
print("Next year you will be", age + 1)
```



Fix: convert the answer right away

```
age = int(input("Your age? "))
print("Next year you will be", age + 1)
```

OUTPUT

```
Your age? 20
Next year you will be 21
```



Fix: convert the answer right away

```
age = int(input("Your age? "))
print("Next year you will be", age + 1)
```

OUTPUT

```
Your age? 20
Next year you will be 21
```

Read it **inside out**: first `input`, then `int`, then store in age.



f-strings: fill in the blanks



Hi _____! Your _____ is ready.

- A message with **blanks**, like a form letter

f-strings: fill in the blanks



Hi **Dana**! Your **pizza** is ready.

- A message with **blanks**, like a form letter
- Python fills each blank with a variable's value

f-strings: fill in the blanks



Hi **Dana**! Your **pizza** is ready.

- A message with **blanks**, like a form letter
- Python fills each blank with a variable's value
- In Python this is called an **f-string**

Your first f-string

```
name = "Dana"
food = "pizza"
age = 20
print(f"Hi {name}! Your {food} is ready.")
print(f"Next year you will be {age + 1}")
```

Your first f-string

```
name = "Dana"
food = "pizza"
age = 20
print(f"Hi {name}! Your {food} is ready.")
print(f"Next year you will be {age + 1}")
```

OUTPUT

```
Hi Dana! Your pizza is ready.
Next year you will be 21
```

Your first f-string

```
name = "Dana"
food = "pizza"
age = 20
print(f"Hi {name}! Your {food} is ready.")
print(f"Next year you will be {age + 1}")
```

OUTPUT

```
Hi Dana! Your pizza is ready.
Next year you will be 21
```

- An `f` right before the opening quote
- Each blank is a name in curly braces: `{name}`
- Numbers and math work too: `{age + 1}`, no `str()` needed

Money needs 2 digits: `:.2f`

```
price = 12.5
print(f"Price: {price:.2f} ILS")
print(10 / 3)
print(f"{10 / 3:.2f}")
```

Money needs 2 digits: `:.2f`

```python
price = 12.5
print(f"Price: {price:.2f} ILS")
print(10 / 3)
print(f"{10 / 3:.2f}")
```

OUTPUT

```
Price: 12.50 ILS
3.3333333333333335
3.33
```

Money needs 2 digits: `:.2f`

```python
price = 12.5
print(f"Price: {price:.2f} ILS")
print(10 / 3)
print(f"{10 / 3:.2f}")
```

OUTPUT

```
Price: 12.50 ILS
3.3333333333333335
3.33
```

`:.2f` = “show exactly 2 digits after the dot”. It rounds for display.

Example: split the pizza bill

```
pizza_price = float(input("Price? "))
num_friends = int(input("Friends? "))
each = pizza_price / num_friends
print(f"Each pays {each:.2f} ILS")
```



Example: split the pizza bill

```
pizza_price = float(input("Price? "))
num_friends = int(input("Friends? "))
each = pizza_price / num_friends
print(f"Each pays {each:.2f} ILS")
```



OUTPUT

```
Price? 160
Friends? 3
Each pays 53.33 ILS
```

Example: split the pizza bill

```
pizza_price = float(input("Price? "))
num_friends = int(input("Friends? "))
each = pizza_price / num_friends
print(f"Each pays {each:.2f} ILS")
```

OUTPUT

```
Price? 160
Friends? 3
Each pays 53.33 ILS
```



Friends: `int`. Price: `float`.

Try it!

Try it in Colab notebooks/L03/L03_02_fstrings.ipynb

1. Ask for a name and a birth year. Print the age.
2. Pizza bill: also print the price with `:.2f`
3. More practice (tip calculator, triangle, `sep` / `end`): `L03_05`



Predict the output

Predict the output!

```
x = 7
print(f"{x} / 2 = {x / 2}")
print(f"{2 / 3:.2f}")
```



Predict the output

Predict the output!

```
x = 7
print(f"{x} / 2 = {x / 2}")
print(f"{2 / 3:.2f}")
```

Answer

```
7 / 2 = 3.5
0.67
```

Outside `{ }` is plain text. `:.2f` rounds 0.666… up to 0.67.



Topic 3

Intro to conditions

Lecture 3 • Input, Output & Conditions



TODAY

1. Talking with the user
2. Converting types and f-strings
3. **Intro to conditions**

Yes-or-no questions

Symbol	Asks...	Example
`> <`	bigger? smaller?	`5 > 3` → `True`
`>=`	bigger or equal?	`18 >= 21` → `False`
`<=`	smaller or equal?	`45 <= 50` → `True`
`==`	equal?	`2 == 2` → `True`
`!=`	not equal?	`14 != 12` → `True`



Yes-or-no questions

Symbol	Asks…	Example
`>` `<`	bigger? smaller?	`5 > 3` → `True`
`>=`	bigger or equal?	`18 >= 21` → `False`
`<=`	smaller or equal?	`45 <= 50` → `True`
`==`	equal?	`2 == 2` → `True`
`!=`	not equal?	`14 != 12` → `True`



Every comparison answers True or False.

Comparisons in Python

```
print(5 > 3)
print(18 >= 21)
print("cat" == "Cat")
print(14 != 12)
```

Comparisons in Python

```
print(5 > 3)
print(18 >= 21)
print("cat" == "Cat")
print(14 != 12)
```

OUTPUT

```
True
False
False
True
```

Comparisons in Python

```
print(5 > 3)
print(18 >= 21)
print("cat" == "Cat")
print(14 != 12)
```

OUTPUT

```
True
False
False
True
```

Line 3: capital letters matter, so the two words are different.

= versus ==

= means “store”

```
grade = 100
```

Puts 100 into the box.



= versus ==

= means “store”

```
grade = 100
```

Puts 100 into the box.



== means “are they equal?”

```
grade == 100
```

Asks a question. Answer: `True`.



= versus ==

= means “store”

```
grade = 100
```

Puts 100 into the box.



== means “are they equal?”

```
grade == 100
```

Asks a question. Answer: `True`.



One = tells. Two == ask.

Chaining: “between” in one line

```
age = 23
print(18 <= age <= 30)
age = 45
print(18 <= age <= 30)
```

Youth discount

Ages 18 to 30 get a cheaper ticket.

Read it like math class:
“age is between 18 and 30”.



Chaining: “between” in one line

```
age = 23
print(18 <= age <= 30)
age = 45
print(18 <= age <= 30)
```

OUTPUT

```
True
False
```

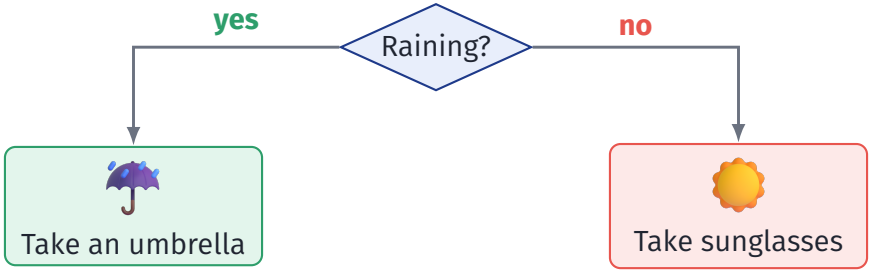
Youth discount

Ages 18 to 30 get a cheaper ticket.

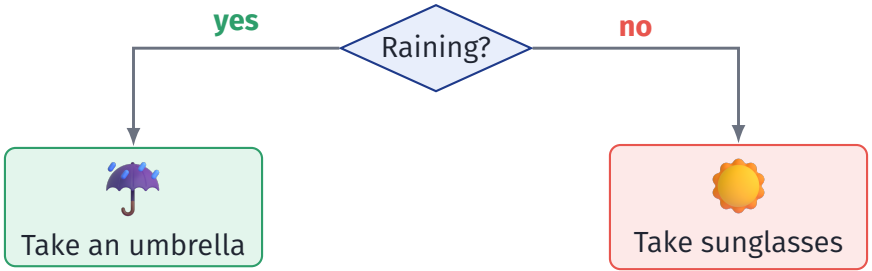
Read it like math class:
“age is between 18 and 30”.



Decisions in real life



Decisions in real life



- A question with a True/False answer
- One action for **yes**, another for **no**

`if` / `else`: who gets into the party?

```
age = int(input("Your age? "))
if age >= 18:
    print("Welcome to the party!")
else:
    print("Sorry, too young")
print("Bye!")
```

`if` / `else`: who gets into the party?

```
age = int(input("Your age? "))
if age >= 18:
    print("Welcome to the party!")
else:
    print("Sorry, too young")
print("Bye!")
```

OUTPUT

```
Your age? 20
Welcome to the party!
Bye!
```

`if` / `else`: who gets into the party?

```
age = int(input("Your age? "))
if age >= 18:
    print("Welcome to the party!")
else:
    print("Sorry, too young")
print("Bye!")
```

OUTPUT

```
Your age? 20
Welcome to the party!
Bye!
```

OUTPUT

```
Your age? 16
Sorry, too young
Bye!
```

`if` / `else`: who gets into the party?

```
age = int(input("Your age? "))
if age >= 18:
    print("Welcome to the party!")
else:
    print("Sorry, too young")
print("Bye!")
```

OUTPUT

```
Your age? 20
Welcome to the party!
Bye!
```

OUTPUT

```
Your age? 16
Sorry, too young
Bye!
```

Only **one** of the two paths runs. `Bye!` always runs.

Indentation means “belongs to”

```
temperature = 34
if temperature > 30:
    print("Bring water!")
print("Have a nice day")
```



Indentation means “belongs to”

```
temperature = 34
if temperature > 30:
    print("Bring water!")
print("Have a nice day")
```

OUTPUT

```
Bring water!
Have a nice day
```



Indentation means “belongs to”

```
temperature = 34
if temperature > 30:
    print("Bring water!")
print("Have a nice day")
```

OUTPUT

```
Bring water!
Have a nice day
```

- Colon : at the end of the **if** line
- 4 spaces in = belongs to the **if**



Indentation means “belongs to”

```
temperature = 34
if temperature > 30:
    print("Bring water!")
print("Have a nice day")
```

OUTPUT

```
Bring water!
Have a nice day
```

- Colon : at the end of the **if** line
- 4 spaces in = belongs to the **if**
- Back to the left = always runs



Example: free shipping from 200 ILS

```
cart_total = float(input("Cart total in ILS? "))
if cart_total >= 200:
    print("Free shipping!")
else:
    missing = 200 - cart_total
    print(f"Add {missing:.2f} ILS for free shipping")
```

Example: free shipping from 200 ILS

```python
cart_total = float(input("Cart total in ILS? "))
if cart_total >= 200:
    print("Free shipping!")
else:
    missing = 200 - cart_total
    print(f"Add {missing:.2f} ILS for free shipping")
```

OUTPUT

```
Cart total in ILS? 150
Add 50.00 ILS for free shipping
```

Example: free shipping from 200 ILS

```
cart_total = float(input("Cart total in ILS? "))
if cart_total >= 200:
    print("Free shipping!")
else:
    missing = 200 - cart_total
    print(f"Add {missing:.2f} ILS for free shipping")
```

OUTPUT

```
Cart total in ILS? 150
Add 50.00 ILS for free shipping
```

OUTPUT

```
Cart total in ILS? 230
Free shipping!
```

Example: even or odd?

```
number = int(input("A number? "))
if number % 2 == 0:
    print(f"{number} is even")
else:
    print(f"{number} is odd")
```



Example: even or odd?

```
number = int(input("A number? "))
if number % 2 == 0:
    print(f"{number} is even")
else:
    print(f"{number} is odd")
```



OUTPUT

```
A number? 7
7 is odd
```

Example: even or odd?

```python
number = int(input("A number? "))
if number % 2 == 0:
    print(f"{number} is even")
else:
    print(f"{number} is odd")
```

OUTPUT

```
A number? 7
7 is odd
```



Even = nothing left over when you divide by 2.

Try it!

Try it in Colab notebooks/L03/L03_03_first_if.ipynb

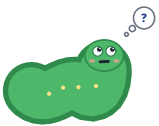
1. Ask for a grade. 56 or more: `Passed!` Else: `Next time!`
2. Ask for the temperature. Above 25: `Shorts`. Else: `Jeans`.
3. Ask for a password. Is it `"python123"`?



Predict the output

Predict the output!

```
temperature = 25
if temperature > 30:
    print("Beach!")
else:
    print("Coffee shop")
print("Done")
```



Predict the output

Predict the output!

```
temperature = 25
if temperature > 30:
    print("Beach!")
else:
    print("Coffee shop")
print("Done")
```

Answer

`Coffee shop`
`Done`

25 is not above 30, so the `else` part runs. `Done` always runs.



Bug Zoo: forgot to convert

BUG ZOO

```
age = input("Your age? ")
print(age + 1)
```

ERROR

```
TypeError: can only
concatenate str (not "int")
to str
```

Text + number

`age` is the text `"20"`, not the number 20.

Fix:

```
age = int(input("Your age? "))
```

Bug Zoo: text that is not a whole number

BUG ZOO

```
hours = int(input("Hours? "))
```

The user typed `3.5`

ERROR

```
ValueError: invalid literal for
int() with base 10: '3.5'
```

Fix: use `float(...)` for decimals

```
print(int("twenty"))
```

ERROR

```
ValueError: invalid literal for
int() with base 10: 'twenty'
```

Fix: `int()` reads digits only



“Invalid literal” = “I cannot read this as a number”.

Bug Zoo: the `if` line

BUG ZOO

```
age = 20
if age >= 18
    print("Welcome!")
```

ERROR

```
SyntaxError: expected ':'
```

Fix: add the colon: `if age >= 18:`

```
age = 20
if age >= 18:
print("Welcome!")
```

ERROR

```
IndentationError: expected an
indented block after 'if'
statement on line 2
```

Fix: push `print` 4 spaces to the right

Bug Zoo: = instead of ==

BUG ZOO

```
age = 20
if age = 18:
    print("Exactly 18!")
```

ERROR

```
SyntaxError: invalid syntax.
Maybe you meant '==' or ':='
instead of '='?
```

Store vs. ask

Inside an `if` you ask a question, so use ==.

Fix: `if age == 18:`

(Ignore the := part.)



Bug Zoo: comparing text with a number

BUG ZOO

```
age = input("Your age? ")
if age >= 18:
    print("Welcome")
```

ERROR

```
TypeError: '>=' not supported between instances of 'str' and 'int'
```

input() gives text

`"20"` is text, `18` is a number. The intern is confused.

Fix:

```
age = int(input("Your age? "))
```

Bug Zoo: “Yes ” is not “yes”

BUG ZOO

```
answer = input("Coffee? ")
if answer == "yes":
    print("Great!")
else:
    print("OK, maybe later")
```

User types `Yes ` (capital Y, extra space):

OUTPUT

```
OK, maybe later
```

No error, wrong answer

To Python, `"Yes "` and `"yes"` are different text.

Fix: clean the answer first:

```
answer = answer.strip().lower()
```

`.strip()` drops spaces at the ends, `.lower()` makes it lowercase.

AI Corner: break my program

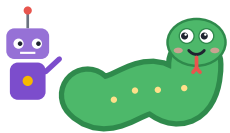
AI CORNER

A good prompt

“Here is my pizza-bill program. What could a user type that would crash it? Explain simply, don’t fix it.” + your code

Double-check

Try each answer yourself: type `abc`, `0`, `-50`. Did it really crash?



Lecture 8: how to catch these crashes.

Recap

✓ Today you learned

- **`print`** shows several values. `input()` always gives text.
- Convert with `int()`, `float()`, `str()`. Build sentences with `f"..."`.
- Comparisons give `True`/`False`. One = stores, two == ask.
- **`if`**/**`else`** picks a path. Indentation means “belongs to”.



Next time: smarter decisions

- More than two paths: `elif`
- Combine rules with `and`, `or`, `not`
- Peek inside: how computers think in 0 and 1





3D emoji: Microsoft Fluent Emoji (MIT License). Pip the snake: original drawing. Full list: images/CREDITS.md